

ĐỀ KIỂM TRA CUỐI KHÓA

CODEBASE - PYTHON 2026

Đề số 02

Câu 1: Tính cước phí vận chuyển

1.1. Mô tả bài toán

Để chuẩn bị cho talkshow diễn ra thành công mỹ mãn, ban Học thuật đóng vai trò là những nhân sự hậu cần, tất bật chuẩn bị yếu phẩm chạy talkshow. Để tiết kiệm chi phí chuẩn bị, ban Học thuật đã sẵn sale các dụng cụ, vật phẩm chủ yếu thông qua sàn thương mại điện tử online.

Là một thành viên ưu tú, bạn hãy giúp ban Học thuật xây dựng hệ thống tính cước phí vận chuyển hàng hóa của đơn vị chuyển phát nhanh này. Cước phí được tính **lũy tiến theo trọng lượng** (kg) như sau:

- **Mức 1:** 5 kg đầu tiên, giá là **10.000 VNĐ/kg**.
- **Mức 2:** Từ kg thứ 6 đến kg thứ 20, giá là **7.500 VNĐ/kg**.
- **Mức 3:** Từ kg thứ 21 trở đi, giá là **5.000 VNĐ/kg**.

1.2. Yêu cầu

Viết hàm `ting_cuoc_van_chuyen(kg)` thực hiện:

1. Nhận vào tham số `kg` (kiểu số thực).
2. Sử dụng cấu trúc `if...elif...else` để tính tổng cước phí trước thuế.
3. Cộng thêm **8% thuế VAT** vào tổng cước phí.
4. Trả về kết quả cuối cùng là số thực, **làm tròn đúng 4 chữ số thập phân**.

1.3. Khung mã nguồn gợi ý

```
def ting_cuoc_van_chuyen(kg):  
    # Bước 1: Kiểm tra điều kiện kg <= 0  
    if kg <= 0:  
        return 0.0000  
  
    # Bước 2: Tính toán theo từng mức (if ... elif ... else)  
    # Viết code ở đây ...  
  
    return result
```

1.4. Test Case mẫu

Input	Output
30	175500.0000

Giải thích:

$$\begin{aligned} & 5 \times 10.000 + 15 \times 7.500 + 10 \times 5.000 \\ &= 50.000 + 112.500 + 50.000 \\ &= 212.500 \text{ VNĐ (trước phụ phí)} \end{aligned}$$

Tổng sau phụ phí (8%):

$$212.500 \times 1,08 = 229.500 \text{ VNĐ}$$

Lưu ý: Đối với test case trên, kết quả 229500.0000 được làm tròn đúng 4 chữ số thập phân. Hàm cần xử lý đúng cho mọi giá trị kg dương, kể cả các giá trị thập phân như 7.5, 12.3, v.v.

Câu 2: Lọc và thống kê chuỗi ký tự

2.1. Mô tả bài toán

Bạn nhận được một **tuple** chứa hỗn hợp các phần tử – bao gồm cả chuỗi ký tự và các kiểu dữ liệu khác. Nhiệm vụ của bạn là lọc ra các chuỗi hợp lệ, tức là các phần tử thỏa mãn **đồng thời** hai điều kiện:

- Phần tử phải là kiểu **str** và không rỗng (sau khi đã `.strip()`).
- Độ dài chuỗi (sau khi `.strip()`) phải **từ 3 đến 15 ký tự**.

2.2. Yêu cầu bài toán

Viết hàm `loc_chuoi_hop_le(du_lieu)` thực hiện các công việc sau:

- Duyệt qua từng phần tử trong **tuple** đầu vào.
- Sử dụng khối `try...except` để cố gắng gọi `.strip()` trên từng phần tử.
- Nếu phần tử không phải **str** (gây ra `AttributeError`), bỏ qua phần tử đó.
- Nếu chuỗi hợp lệ (không rỗng và độ dài từ 3 đến 15), thêm vào danh sách kết quả ở dạng đã `.strip()` và viết hoa chữ cái đầu (`.capitalize()`).

2.3. Kết quả trả về

Hàm cần trả về một **Dictionary** chứa hai thông tin:

1. "chuoi_hop_le": Danh sách các chuỗi đã được lọc và chuẩn hóa.
2. "trung_binh_do_dai": Độ dài trung bình của các chuỗi hợp lệ (làm tròn 4 chữ số thập phân). Nếu không có chuỗi hợp lệ nào, giá trị này bằng 0.

2.4. Khung mã nguồn gợi ý

```
def loc_chuoi_hop_le(du_lieu: tuple) -> dict:
    ket_qua = []
    for item in du_lieu:
        try:
            # Co gang goi .strip() tren phan tu
            # Viet code o day ...

            # Kiem tra dieu kien do dai tu 3 den 15
            # Viet code o day ...

            # Neu khong phai str thi bo qua
        except AttributeError:
            pass

    # Tinh trung binh do dai va tra ve dictionary
    # Viet code o day ...
    return {...}
```

2.5. Test Case mẫu

Input:

```
(" python ", "ok", 42, " ", "hello world", None,
 "AI", "deep learning rocks", True, " numpy ")
```

Output:

```
{
    "chuoi_hop_le": ["Python", "Hello world", "Numpy"],
    "trung_binh_do_dai": 7.3333
}
```

Giải thích từng phần tử:

- " python " → Hợp lệ (6 ký tự) → "Python"
- "ok" → Không hợp lệ (2 ký tự < 3), bỏ qua
- 42 → AttributeError, bỏ qua
- " " → Rỗng sau strip(), bỏ qua

- "hello world" → Hợp lệ (11 ký tự) → "Hello world"
- None → AttributeError, bỏ qua
- "AI" → Không hợp lệ (2 ký tự < 3), bỏ qua
- "deep learning rocks" → Không hợp lệ (20 ký tự > 15), bỏ qua
- True → AttributeError, bỏ qua
- " numpy " → Hợp lệ (5 ký tự) → "Numpy"

Trung bình độ dài: $(6 + 11 + 5) \div 3 = 22 \div 3 \approx 7.3333$

Lưu ý: "hello world" tính là 11 ký tự (có dấu cách ở giữa).

Câu 3: Quản lý danh sách người tham dự Talkshow

3.1. Mô tả bài toán

Sự kiện Talkshow "Tech Career: Stay Local or Go Global" do NITC tổ chức thu hút rất nhiều người quan tâm và tham dự. Tuy nhiên, một số sinh viên đã điền đơn đăng ký tham dự Talkshow nhưng lại không tới tham dự, hoặc chỉ checkin thời gian đầu nhưng không kiên nhẫn ngồi đến phút cuối checkout, dẫn đến không đạt đủ điều kiện quyền lợi điểm Đoàn từ sự kiện.

Nhiệm vụ của bạn là xây dựng hệ thống quản lý danh sách sinh viên tham dự sự kiện Talkshow này của NITC. Dữ liệu được lưu trong file `c3_input.csv`. Hãy đọc dữ liệu từ file, xác định điều kiện tham dự hợp lệ của từng sinh viên, rồi xuất báo cáo ra file kết quả.

3.2. Yêu cầu xây dựng Lớp (Object Oriented Programming)

Xây dựng lớp `SinhVien` để quản lý thông tin sinh viên với các thành phần sau:

- **Thuộc tính (Attributes):** `msv` (mã sinh viên), `ho_ten` (họ tên, kiểu chuỗi), `nganh` (ngành học, kiểu chuỗi), `checkin` (kiểu boolean), `checkout` (kiểu boolean).
- **Phương thức khởi tạo (`__init__`):** Gán giá trị cho các thuộc tính khi tạo đối tượng.
- **Phương thức `kiem_tra_dieu_kien()`:** Trả về chuỗi "Dat dieu kien" nếu cả `checkin` lẫn `checkout` đều là `True`, ngược lại trả về "Khong dat".

3.3. Yêu cầu xử lý Tập tin (File Handling)

Viết hàm `doc_sinh_vien_tu_file(ten_file_nhap)` thực hiện:

1. Mở tập tin có tên `ten_file_nhap` ở chế độ đọc ('r') với bảng mã utf-8.
2. Giả sử mỗi dòng dữ liệu trong file có cấu trúc:
`MSV, HoTen, Nganh, Checkin, Checkout`
Trong đó `Checkin` và `Checkout` nhận giá trị `True` hoặc `False` (dạng chuỗi). Chuyển đổi mỗi dòng thành một đối tượng `SinhVien`, rồi trả về danh sách tất cả các đối tượng.
3. **Xử lý ngoại lệ:** Dùng `try...except` để bắt lỗi `FileNotFoundError` và in thông báo nếu không tìm thấy file.

Viết hàm `ghi_bao_cao(danh_sach_sv, ten_file_xuat)` thực hiện:

1. Nhận vào một danh sách các đối tượng thuộc lớp `SinhVien`.
2. Mở tập tin có tên `ten_file_xuat` ở chế độ ghi ('w') với bảng mã utf-8.
3. Với mỗi sinh viên, ghi một dòng vào file theo định dạng:
`MSV - HoTen - Nganh - KetQua`
Trong đó `KetQua` là kết quả trả về từ phương thức `kiem_tra_dieu_kien()`.
4. **Xử lý ngoại lệ:** Dùng `try...except` để bắt lỗi `IOError` và in thông báo nếu không thể ghi file.

3.4. Khung mã nguồn gợi ý

```
class SinhVien:
    def __init__(self, msv, ho_ten, nganh, checkin, checkout):
        self.msv = msv
        self.ho_ten = ho_ten
        self.nganh = nganh
        self.checkin = checkin
        self.checkout = checkout

    def kiem_tra_dieu_kien(self):
        # Tra ve "Dat dieu kien" neu ca checkin va checkout la
        # True
        # Viet code o day ...
        return ...

def doc_sinh_vien_tu_file(ten_file_nhap: str) -> list:
    ds_doi_tuong = []
    try:
        # Doc file input va xu ly tung dong
        # Luu y: chuyen "True"/"False" (string) sang boolean
```

```

        # Viet code o day ...
        return ds_doi_tuong
    except FileNotFoundError:
        print(f"Loi: Khong tim thay file {ten_file_nhap}")
        return []

def ghi_bao_cao(danh_sach_sv: list, ten_file_xuat: str):
    try:
        # Xu ly tung sinh vien va ghi ra ten_file_xuat
        # Viet code o day ...
        print(f"Da ghi bao cao vao file {ten_file_xuat}")
    except IOError:
        print("Loi: Khong the ghi du lieu vao file!")

# (OPTIONAL) Chuong trinh chinh de test
if __name__ == "__main__":
    file_vao = "c3_input.csv"
    file_ra = "c3_output.txt"

    # Buoc 1: Doc tu file de tao danh sach doi tuong
    danh_sach = doc_sinh_vien_tu_file(file_vao)

    # Buoc 2: Neu co du lieu thi ghi bao cao
    if danh_sach:
        ghi_bao_cao(danh_sach, file_ra)

```

3.5. Test Case mẫu

Input: file c3_input.csv

```

SV001,Nguyen Van An,CNTT,True,True
SV002,Tran Thi Bich,QTKD,True,False
SV003,Le Minh Chau,KTPM,False,False

```

Output: file c3_output.txt

```

SV001 - Nguyen Van A - CNTT - Dat dieu kien
SV002 - Tran Thi B - KHMT - Khong dat
SV003 - Le Minh C - TTNT - Khong dat

```

Giải thích:

MSV	Họ tên	Ngành	^c Checkin	Checkout	Kết quả
SV001	Nguyen Van An	CNTT	True	True	Dat dieu kien
SV002	Tran Thi Bich	QTKD	True	False	Khong dat
SV003	Le Minh Chau	KTPM	False	False	Khong dat

Lưu ý: Khi đọc file, chuỗi "True"/"False" cần được chuyển sang kiểu bool trước khi gán vào thuộc tính (gợi ý: `checkin = val.strip() == "True"`).

Câu 4: Phân tích dữ liệu nhiệt độ với NumPy và Pandas

4.1. Mô tả bài toán

Một trạm khí tượng ghi lại nhiệt độ trung bình (đơn vị: °C) theo từng ngày trong tháng. Bạn cần sử dụng **NumPy** và **Pandas** để phân tích dữ liệu này và phân loại từng ngày theo mức nhiệt độ.

Cho dữ liệu đầu vào gồm:

- `ngay`: Danh sách các ngày đo (dạng chuỗi).
- `nhiet_do`: Danh sách nhiệt độ tương ứng (đơn vị: °C).

Ví dụ:

```
ngay = ["2024-07-01", "2024-07-02", "2024-07-03", "2024-07-04"]
nhiet_do = [38.5, 27.0, 41.2, 33.6]
```

4.2. Yêu cầu thực hiện

Viết hàm `phan_tich_nhiet_do(ngay, nhiet_do)` thực hiện các công việc sau:

1. Sử dụng NumPy để:

- Chuyển danh sách `nhiet_do` (list) thành mảng NumPy (ndarray).
- Tính:
 - Nhiệt độ trung bình.
 - Nhiệt độ cao nhất.
 - Nhiệt độ thấp nhất.
 - Độ lệch chuẩn (`np.std`).

2. Sử dụng Pandas để:

- Tạo một DataFrame gồm hai cột: "Ngày" và "NhiệtDo".
- Thêm cột mới "PhanLoai" theo quy tắc:
 - "Lạnh" nếu nhiệt độ < 20
 - "Mát" nếu nhiệt độ từ 20 đến dưới 35
 - "Nóng" nếu nhiệt độ ≥ 35
- Gợi ý: Thiết kế hàm `phan_loai` rồi dùng `.apply(phan_loai)` cho cột mới.

3. Trả về một Dictionary chứa:

- "trung_binh"
- "cao_nhat"
- "thap_nhat"
- "do_lech_chuan"
- "bang_du_lieu" (DataFrame kết quả)

4.3. Khung mã nguồn gợi ý

```
import numpy as np
import pandas as pd

def phan_tich_nhiet_do(ngay, nhiet_do):
    # Buoc 1: Chuyen nhiet_do thanh mang NumPy
    arr = np.array(nhiet_do)

    # Buoc 2: Tinh cac thong ke co ban
    # Viet code o day ...

    # Buoc 3: Tao DataFrame bang Pandas
    df = pd.DataFrame({
        "Ngay" : ngay,
        "NhietDo": nhiet_do
    })

    # Buoc 4: Them cot PhanLoai
    # Viet code o day ...

    # Buoc 5: Tra ve ket qua
    return {
        "trung_binh" : round(trung_binh, 4),
        "cao_nhat" : round(cao_nhat, 4),
        "thap_nhat" : round(thap_nhat, 4),
        "do_lech_chuan" : round(do_lech_chuan, 4),
        "bang_du_lieu" : df
    }

# (Optional) Chuong trinh thuc thi chay thu
if __name__ == "__main__":
    ngay = ["2024-07-01", "2024-07-02", "2024-07-03", "2024-07-04"]
    nhiet_do = [38.5, 27.0, 41.2, 33.6]

    ket_qua = phan_tich_nhiet_do(ngay, nhiet_do)

    print("Trung binh :", ket_qua["trung_binh"])
    print("Cao nhat :", ket_qua["cao_nhat"])
    print("Thap nhat :", ket_qua["thap_nhat"])
    print("Do lech chuan:", ket_qua["do_lech_chuan"])
    print("\nBang du lieu:")
    print(ket_qua["bang_du_lieu"])
```

4.4. Test Case mẫu

Input:

```
["2024-07-01", "2024-07-02", "2024-07-03", "2024-07-04"]
```



```
[38.5, 27.0, 41.2, 33.6]
```

Output:

```
{
  "trung_binh"    : 35.075,
  "cao_nhat"      : 41.2,
  "thap_nhat"     : 27.0,
  "do_lech_chuan": 5.3997,
  "bang_du_lieu"  : ...
}
```

DataFrame kết quả:

Ngày	NhietDo	PhanLoai
2024-07-01	38.5	Nong
2024-07-02	27.0	Mat
2024-07-03	41.2	Nong
2024-07-04	33.6	Mat

Giải thích:

- Trung bình: $(38,5 + 27,0 + 41,2 + 33,6) \div 4 = 140,3 \div 4 = 35,075$
- Độ lệch chuẩn: `np.std([38.5, 27.0, 41.2, 33.6])` $\approx 5,3997$
- Phân loại: $38,5 \geq 35 \rightarrow \text{Nong}$; $27,0 \in [20, 35) \rightarrow \text{Mat}$; $41,2 \geq 35 \rightarrow \text{Nong}$; $33,6 \in [20, 35) \rightarrow \text{Mat}$.